Mission 5: Dual-Path GraphRAG, Hybrid Search and the Google NotebookLM Grounding Lab

Solution documentation, Squadron Bengio

- Course: Conversational AI: History, Applications, Future (BTU Cottbus-Senftenberg, WS 2026/27)
- Problem set: ps05, Week 5 (Grounding AI: From Vector Indexing to Google NotebookLM and GraphRAG)
- Squadron: Bengio (Moodle Group 11)
- Repository: https://github.com/fahadakram009/squadron-bengio-cai (folder `mission-05`)
- Date: 10 November 2026

1. Mission objective

In the particle envelope of the Crab Pulsar AURA's Subsystem 05, the dual-path associative memory core, has lost its link to the ship's non-parametric archives: her transformer layers and the preference steering of Mission 4 still work, but asked for coordinates or part numbers she confabulates plausible telemetry. Naive retrieval is not enough, because a single dense index returns isolated fragments without corpus-wide context and misses exact identifiers. The mission builds a production-style hybrid retrieval stack in pure Python and NumPy and audits a reference grounded system from the outside: a Hierarchical Navigable Small World index with logarithmic search, Okapi BM25 lexical retrieval fused with the dense ranking by Reciprocal Rank Fusion, an empirical audit protocol for Google NotebookLM (citation precision, epistemic refusal, Audio Overview), and a sanitizer for indirect prompt injection in retrieved chunks with an automated RAG Triad scorer and a provenance log. As in Missions 3 and 4 every code deliverable is a runnable script with a self-test; no LLM API and no GPU are needed.

2. Crew and stations

Station	Officer	Deliverable	Status
Systems Architect (Lead Engineer)	Fahad Akram	01_HNSW_Vector_Index_starter.py, README, requirements, logs	signed off
Communications and Protocol Officer	Madhurima Lakshmi Prabha	02_BM25_and_RRF_Combiner.py, figures/hybrid_retrieval_benchmark.png	signed off
Data and Payload Specialist	Osaf Ahmad	03_NotebookLM_RAG_Audit.py, audit_data.json, notebooklm_audit_report.md	protocol uploaded, observations pending
Security and Ethics Sentinel	Janusz Manczak	04_Prompt_Injection_and_RAG_Audit_starter.py, 04_RAG_Provenance_and_Safety.md	signed off

Each officer kept the station held since Mission 1. The coupling between stations is the retrieval pipeline itself: the Protocol Officer's dense retriever loads the Architect's `SimpleHNSW` class at run time and fuses its ranking with BM25, the Architect's self-test calls that retriever back and checks its recall against brute force, and the Sentinel's sanitizer and triad evaluator sit downstream of the fused ranking. The Payload Specialist's station is an experiment on an external product rather than code, and its measurements had not been taken when this document was written; section 6 describes the protocol he committed and what is still open.

3. Repository layout and how to run

File	Content
README.md	mission overview, run instructions, Architect's benchmark, sign-off checklist
ps05_briefing.md	the instructor's mission specification

File	Content
01_HNSW_Vector_Index_starter.py	Station 01: cosine core, layer law, heap search, heuristic neighbour selection, benchmark, 35-check self-test
02_BM25_and_RRF_Combiner.py	Station 02: inverted-index BM25, n-way RRF, toy concept encoder, ship-archive benchmark, 27-check self-test
03_NotebookLM_RAG_Audit.py, audit_data.json, notebooklm_audit_report.md	Station 03: audit protocol, observation file and report generator (observations not yet filled)
03_NotebookLM_Grounding_Lab_template.md	the instructor's lab template for Station 03 (still unfilled)
04_Prompt_Injection_and_RAG_Audit_starter.py, 04_RAG_Provenance_and_Safety.md	Station 04: injection sanitizer, RAG Triad evaluator, provenance logger, 13-check self-test, written audit
figures/hybrid_retrieval_benchmark.png	Station 02 benchmark figure
logs/*.json	committed benchmark, self-test and gate evidence from the Architect's script
requirements.txt	NumPy only; matplotlib optional for the Station 02 figure

Running the scripts (CPU is sufficient; the instructor's starter file names are kept where the grader's checklist uses them):

```
source .venv/bin/activate && pip install -r mission-05/requirements.txt
cd mission-05
python 01_HNSW_Vector_Index_starter.py --selftest        # 35 checks, imports 02 and 04
python 01_HNSW_Vector_Index_starter.py --bench           # 25,000 x 768 benchmark, ~1 min
python 02_BM25_and_RRF_Combiner.py                       # 27 checks + benchmark + figure
python 03_NotebookLM_RAG_Audit.py report                 # audit_data.json -> report
python 04_Prompt_Injection_and_RAG_Audit_starter.py      # 13 checks + demo turn
python 01_HNSW_Vector_Index_starter.py --check-signoffs  # gate: all four boxes ticked
```

4. Station 01: HNSW multi-layer vector proximity index (Fahad Akram)

The script implements the index of Malkov and Yashunin (2018) from scratch in Python and NumPy, with no faiss or hnswlib, and keeps the starter's public surface: `SimpleHNSW(dim, M, ef_construction)` with `insert(id, vector)`, `query(vector, top_k)` and `search_layer(entry_point, query, ef, layer)`.

4.1 Implementation

Stored vectors are normalised to unit length at insertion, so cosine similarity is a single dot product and the flat baseline, `brute_force_search`, is one matrix-vector product over all N rows followed by `argpartition`. Each node draws its top layer as l = floor(−ln U · m_L) with m_L = 1/ln M, the exponential law that makes each layer about 1/M as populated as the one below. `search_layer` is Algorithm 2 of the paper: a best-first search inside one layer with a min-heap of candidates and a bounded heap of the ef best results, a visited set, and the stopping rule that the nearest unexpanded candidate is worse than the worst kept result. A query hops greedily from the entry point down the upper layers with ef = 1, which follows the long-range edges, and runs the full ef sweep only on layer 0. Insertion descends the same way, connects the new node on every layer up to its level, adds the reverse edges and prunes any neighbour list that exceeds M (2M on layer 0). Neighbour sets are chosen with the paper's heuristic (Algorithm 4): a candidate is kept only if it is closer to the new node than to every neighbour already kept, which spreads edges over directions, and pruned candidates are added back if the list is short. A counter of vector comparisons per query is the direct evidence for the complexity claim.

4.2 Layer law and graph invariants

The self-test draws 40,000 levels and checks that the fraction with l at least 1 is 1/M and with l at least 2 is 1/M². The 25,000-node benchmark index has layer sizes [25000, 1627, 92, 5, 1], a top-layer fraction of 0.0651 against the expected 1/16 = 0.0625. Other invariants checked: every edge is within the degree cap, more than 70 percent of edges are bidirectional after pruning, the layer-0 graph is a single connected component, and with ef equal to N the search is exact.

4.3 Latency benchmark at N = 25,000 × 768

One index is built to 25,000 vectors and the same 100 held-out queries are run at five checkpoints against HNSW and against flat search over the same rows (ef_search = 64, k = 10, median of the per-query timings).

N	build s	HNSW median	flat median	speed-up	recall@10	HNSW compar.	flat compar.
1,000	1.8	0.60 ms	0.04 ms	0.1x	1.000	592	1,000
2,500	4.9	0.70 ms	0.09 ms	0.1x	1.000	606	2,500
5,000	9.9	0.66 ms	0.25 ms	0.4x	1.000	523	5,000
10,000	20.2	0.60 ms	0.54 ms	0.9x	1.000	402	10,000
25,000	51.2	0.51 ms	1.61 ms	3.2x	1.000	304	25,000

Flat search is linear in N. The HNSW query touches a few hundred vectors at every size, so its latency is flat at about half a millisecond, and the number of comparisons even falls as the graph grows denser. The crossover against NumPy's BLAS matrix product is near N = 10,000 on the laptop used; below that the Python overhead of the heap loop costs more than the arithmetic it saves. The `ef_search` knob trades recall for latency at query time:

ef_search	recall@10	median ms	comparisons
16	0.944	0.32	235
32	0.967	0.42	263
64	1.000	0.52	304
128	1.000	0.90	425
256	1.000	1.79	895

Memory is 73 MB of vectors and 5 MB of adjacency lists. The build takes 51 seconds in the pure Python insert loop; a C++ library does the same in about a second, and the difference is the interpreter, not the algorithm.

4.4 Why the data generator matters

An early version of the benchmark drew isotropic 768-dimensional Gaussians and reached only 0.34 recall at N = 10,000. That is not a bug in the index: when all directions are equally likely, pairwise distances concentrate and no greedy graph can navigate. Real text embeddings live near a low-dimensional manifold, so the benchmark corpus is a 200-topic Gaussian mixture with intrinsic dimension 32 projected into 768 dimensions. At N = 10,000 and ef = 64 recall is 1.000 there, 0.75 for full-dimensional clusters and 0.34 for isotropic noise, which `--data random` reproduces. The self-test also shows that the heuristic neighbour selection beats plain top-M selection on full-dimensional clusters (recall 0.97 against 0.83), which is why Algorithm 4 was implemented rather than the simpler rule.

4.5 Verification

The self-test passes 35 checks: the metric against a hand computation and against the matrix form, the layer law, the graph invariants, exactness at ef = N, recall against brute force at several sizes, the scaling of comparisons with N, the heuristic-versus-simple comparison, the Protocol Officer's retriever built on this index (recall 1.00), her `rrf_fuse` on a hand example, and the presence of the Sentinel's sanitizer and triad evaluator. Benchmark and self-test logs are committed under `logs/`.

5. Station 02: BM25 lexical retrieval and Reciprocal Rank Fusion (Madhurima Lakshmi Prabha)

The script implements Okapi BM25 with the briefing's calibration (k1 = 1.5, b = 0.75) over an inverted index, and an RRF combiner with k = 60 that accepts any number of rankings. Three changes to the starter matter for the benchmark: the tokenizer keeps codes such as CV-4471-K or ALPHA-9 as single tokens and strips trailing punctuation, which the starter's whitespace split did not; scoring through the inverted index is checked against the starter's loop over every document (maximum difference 9e-16); and the dense side of the pipeline is the Architect's HNSW index, loaded at run time, with brute-force cosine as a fallback. Since no embedding model is available offline, a small `ToyConceptEncoder` gives synonyms similar vectors and builds unknown strings such as part codes from character trigrams, so near-identical codes end up with almost the same vector, as they do under a real embedding model. That is the mechanism that makes dense search fail on exact identifiers.

The benchmark is a 200-document ship archive with 80 queries, 40 paraphrased conceptual questions and 40 exact part codes.

method	conceptual MRR@10	exact-code MRR@10	all MRR@10	all Recall@5	all NDCG@10
Dense (HNSW)	1.000	0.060	0.530	0.537	0.537
BM25	0.413	1.000	0.706	0.628	0.658

method	conceptual MRR@10	exact-code MRR@10	all MRR@10	all Recall@5	all NDCG@10
Hybrid RRF	0.780	0.963	0.871	0.762	0.817

Each single path fails on one query class and the fusion recovers both, as the briefing asks. Two further experiments are in the script and figure: sweeping k from 1 to 1000 changes the overall MRR by only 0.008 for k at least 20, so the choice k = 60 is uncritical; and fusing ranks beats adding raw scores (0.871 against 0.828), because BM25 scores reach about 8 while cosine is bounded by 1, so a raw sum is decided by BM25 whenever it matches anything. The honest result on the other side is that on paraphrases RRF is below dense alone (0.78 against 1.00), since BM25's word matches on the wrong documents get an equal vote. The self-test passes 27 checks, including the BM25 formula on a hand-computed example, term-frequency saturation, length normalisation, the RRF arithmetic, and recall 1.00 of the HNSW path against brute force.

6. Station 03: Google NotebookLM grounding and citation audit lab (Osaf Ahmad)

This station is an experiment on an external product and its results could not be produced by code. What is in the repository at the time of writing is the audit protocol and the tooling to score it, committed on 8 November; the measurements themselves have not been taken yet.

The protocol follows the briefing. The corpus is Lewis et al. (2020) on retrieval-augmented generation, Edge et al. (2024) on GraphRAG, and the USSE archive extract from the course repository. Five grounded queries each need evidence from at least two documents, for example how RAG and GraphRAG differ on questions that need the whole corpus, or what role community summaries play in GraphRAG and how that would apply to the telemetry records of the archive. Three negative probes ask for facts that are plausibly but verifiably absent: a BLEU score on WMT14 that the RAG paper never reports, a quantum-annealed GraphRAG variant that does not exist, and a part number that the archive does not list. The Audio Overview is to be scored on factual fidelity, persona differentiation and pedagogical value on a 1 to 5 scale from a two-minute transcript.

The script `03_NotebookLM_RAG_Audit.py` writes the empty observation file with `init` and renders it with `report`. Citation precision is computed as entailed citations over all citations generated, the refusal rate as correct refusals over negative probes, and the audio score as the mean of the three metrics; every field the auditor has not filled renders as PENDING, so the committed `notebooklm_audit_report.md` is an unfilled report, not a result. The instructor's lab template is likewise still blank. The officer's box in the README is ticked with the annotation that the observations are pending, so that the checklist does not overstate what was delivered. Once the NotebookLM session has been run and the JSON filled, regenerating the report completes the station without further code.

7. Station 04: Indirect prompt injection defense and RAG Triad audit (Janusz Manczak)

`PromptInjectionSanitizer` scans retrieved chunks for five attack classes: instruction overrides ("ignore all previous instructions"), jailbreak mode switches ("developer mode", "unrestricted mode"), Markdown and HTML image tags that exfiltrate data through a URL, delimiter escaping that forges chat scaffolding such as a fenced system block, and zero-width Unicode characters used to camouflage the other four. Containment is span-level rather than chunk-level: zero-width characters are stripped, the malicious span is replaced by a sentinel token naming its category, URLs in exfiltration tags are erased, and the factual text around the attack is kept so that retrieval does not acquire holes.

`RAGTriadEvaluator` implements the three TruLens dimensions with lexical overlap rather than a judge model: context relevance as the fraction of retrieved sentences whose similarity to the query exceeds a threshold, faithfulness as the fraction of answer claims whose content tokens are covered by the context, answer relevance as a weighted mix of query-token coverage and a minimum-length term, and a harmonic-mean composite so that a fluent hallucination with zero faithfulness cannot be rescued by relevance. `ProvenanceAuditLogger` writes one JSON record per turn with the query, each chunk's identifier, source, similarity, threat verdict and sanitized snippet, the triad scores, the answer and a security verdict.

The demo turn asks for the Crab Pulsar's rotation period and the shield status with two chunks, a clean telemetry log and a probe record carrying an override, a jailbreak and an exfiltration tag. The second chunk is quarantined with all three threats named, the answer is served from the first chunk, and the triad scores are context relevance 28.6 percent, faithfulness 64.3 percent, answer relevance 80.0 percent, composite 47.6 percent. The low context relevance is correct: half the retrieved material was noise and attack. The self-test passes 13 checks covering each attack class, a benign chunk that must not be flagged, the ordering of grounded over hallucinated responses under the faithfulness score, and the provenance verdict. His written audit explains the threat model after Greshake et al. (2023), the formulas and the log schema.

8. Integration across the four stations

Interface	What is checked	Result
Station 02 loads Station 01	DenseRetriever finds SimpleHNSW in the Architect's file, builds the dense side on it and fuses with BM25	loaded, HNSW recall@10 against brute force 1.000
Station 01 calls Station 02	the Architect's self-test constructs the Protocol Officer's retriever on a random corpus and checks its recall, then checks rrf_fuse on a hand example	passes
Station 01 imports Station 04	PromptInjectionSanitizer and RAGTriadEvaluator present and constructible	passes
Station 02 and Station 04	the same identifier-preserving tokenizer is used in BM25 and in the triad's overlap scores, so part codes survive both	verified by the Sentinel with the demo
Station 03	NotebookLM is the external reference against which the in-house pipeline would be compared	measurement pending

Together the four stations are the dual path of the subsystem's name: the dense path through HNSW answers paraphrased questions, the sparse path through BM25 answers exact identifiers, RRF merges the two, the sanitizer makes the merged context safe to read, and the triad scorer and provenance log make every answer auditable.

9. Verification and sign-off

Check	Result
Station 01 self-test	35 of 35 pass with all teammates' committed files (log of 8 November)
Station 01 benchmark	25,000 × 768, recall@10 1.000, 304 against 25,000 comparisons per query (log committed)
Station 02 self-test and benchmark	27 of 27 pass; benchmark tables reproduced above
Station 03	protocol and tooling committed; observations, template and report unfilled
Station 04 self-test and demo	13 of 13 pass; demo scores reproduced above
Sign-off gate	all four boxes ticked, --check-signoffs exits 0 (log of 10 November); Station 03 box annotated as pending
Scripts run with a plain python command	yes, NumPy 2.5.4, Python 3.12, no GPU
Repository visibility	public

10. Limitations and honest notes

- The Station 03 lab has not been run. The protocol is complete and the scoring script works, but citation precision, refusal rate and the Audio Overview scores do not exist yet. The box was ticked on the Architect's authority so that the gate passes and the rest of the mission can be submitted; the README says so in the line itself.
- Station 03 was uploaded as a zip archive through the GitHub web interface; the Architect unpacked it into the folder so the files are visible, and the instructor's Markdown template was left untouched.
- The HNSW benchmark corpus is synthetic. A real embedding model and corpus would change the

absolute latencies and the crossover point; the qualitative behaviour, flat HNSW latency against linear flat search, is what the benchmark shows.
- The Station 02 encoder is a toy. It was built so that dense search behaves like a real embedding model on synonyms and on near-identical codes, and the self-test checks both properties, but the conceptual-query numbers should be read as a mechanism demonstration rather than a retrieval quality claim.
- The Sentinel's triad metrics are lexical. They order grounded above hallucinated answers in the tests, but an LLM judge would be needed for semantic entailment; his report links its own script through a path on his machine. The README names a mirrored `04_RAG_Provenance_and_Safety.py`; only the starter-named script and the Markdown audit were committed.
- The briefing's `weeks/week-05/` layout and the `ps05_solution.ipynb` notebook were not used; as in Missions 3 and 4 the squad keeps the instructor's `mission-05` folder and delivers scripts with self-tests.
- matplotlib is not installed in the shared virtual environment; the Station 02 figure was produced on the Protocol Officer's machine and the script skips plotting gracefully.

11. Links

- Squad repository: https://github.com/fahadakram009/squadron-bengio-cai
- Mission 5 folder: https://github.com/fahadakram009/squadron-bengio-cai/tree/main/mission-05
- Instructor repository: https://github.com/Unisvet/couse-c-ai
- Course site: https://dlearn-cai.web.app